THE UNIVERSITY OF MELBOURNE

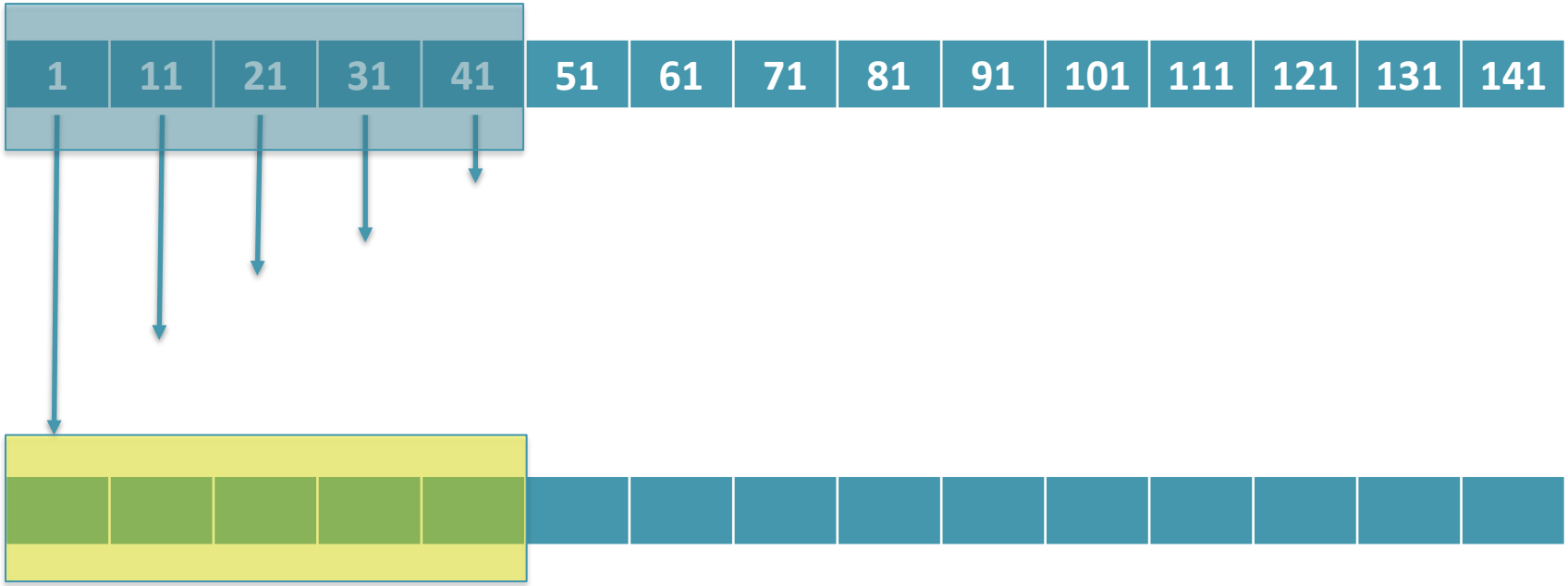
COMP30023 – Computer Systems

Protocol Design Exercise

Dr Lachlan Andrew

©2025 University of Melbourne

Recap: TCP Sliding Window



Remote Procedure Call protocol

- You will design a protocol for remote procedure calls (RPC)
- Step 1: General protocol design considerations
- Step 2: Understand RPC
- Step 3: Design

© 2025 University of Melbourne

Step 1: Protocol design decisions

- What does an interaction look like?
  - What “services” (API) does it expose to the higher layer?
  - What messages/packets/… are sent to achieve that?
  - Connection oriented or connectionless?
- What data needs to be communicated?
  - For the task
  - For the protocol itself
  - What lower layer protocol should it use?
- What format should be used?
  - Headers? Fields? Free-form data? Valid data ranges?
- How are errors handled?

© 2025 University of Melbourne

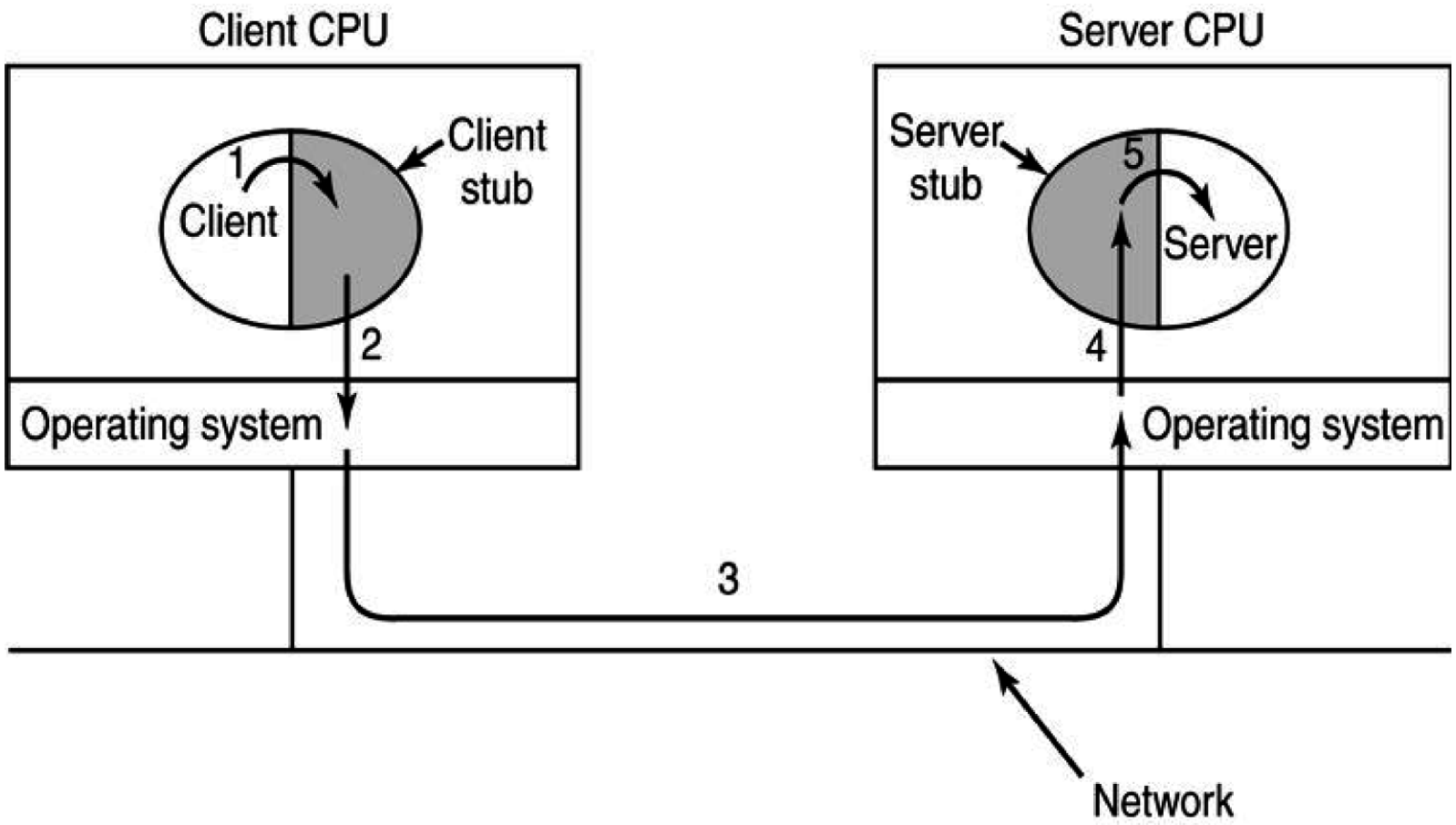
Step 2:
What are Remote Procedure Calls?

- RPC – Remote Procedure Calls
  - Allow calling procedures on a remote server as if they are local to the client
  - Hides the networking aspects from the programmer
- RPC isn't a single protocol/API. Dozens of variants exist.
  - Database APIs, Google Maps API, gRPC, …
- How it works abstractly:
  1. Client process on Machine A calls procedure on Machine B
  2. Thread on machine A is suspended, while execution of the procedure takes place on Machine B
  3. Machine B responds with result to Machine A, which then continues processing

Remote Procedure Calls

- To hide the networking, the client and server must be bound to respective stubs
  - Client stub – operates in the client address space
  - Server stub – operates in the server address space
- From the perspective of the client and server processes all the calls are local
- Parameters can be passed and returned
  - Marshalling – convert the in-memory data structure to a form that can be stored or transmitted
  - Unmarshalling – covert the stored or transmitted data into an in-memory data structure

Remote Procedure Calls



Remote Procedure Calls

Conceptually simple, but many challenges exist.

- Cannot pass pointers easily – client and server are in different address spaces
  - Possible to marshal and unmarshal underlying value and create a pointer in each address space, does not work for complex data structures
- Weakly typed languages like C can present problems, e.g. unknown array sizes
- Unable to deduce parameter types
- Global variables are not shared

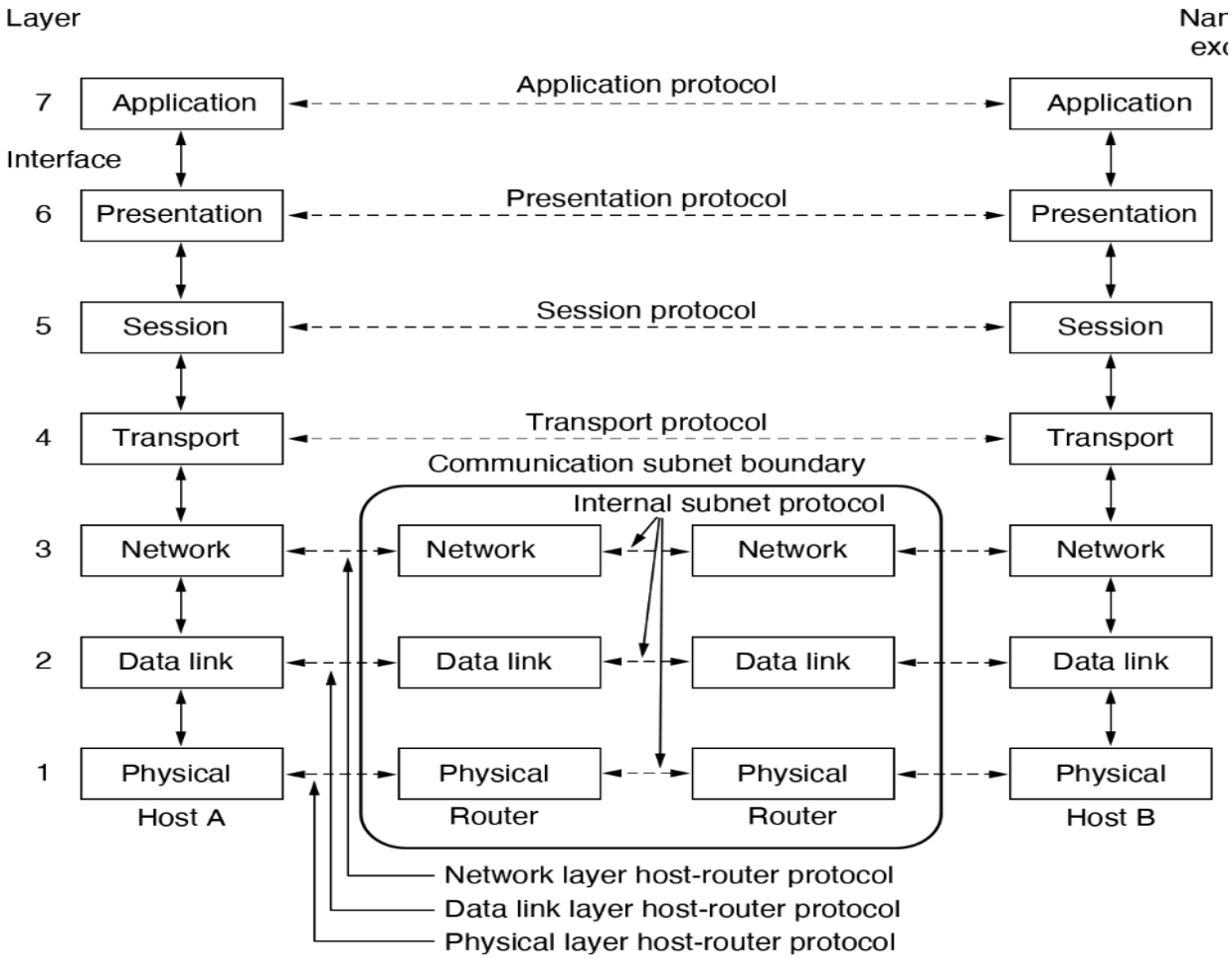
Where does it fit?

```
    // #define required for VSCode
#define _POSIX_C_SOURCE 200112L
#include <netdb.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

int connfd = 0, re = 1, s, n;
struct addrinfo hints, *res, *rp;

memset(&hints, 0, sizeof hints);
hints.ai_family = AF_INET6;
hints.ai_socktype = SOCK_STREAM;
hints.ai_flags = AI_ACTVE;
```

```
s = getaddrinfo(“localhost",
                “5000”, &hints, &res);
for (rp = res; rp != NULL;
            rp = rp->ai_next) {
    connfd = socket(rp->ai_family,
                    rp->ai_socktype,
                    rp->ai_protocol);
    if (connfd == -1) continue;
    if (connect(connfd,
                rp->ai_addr,
                rp->ai_addrlen) != -1
        break;
    close(connfd);
}
write (connfd, “Hello, network!\r\n”);
close (connfd);
```

© 2025 University of Melbourne

What position in the stack?



- Use data
- Tidy up e2e
- Get data e2e
- Tidy up p2p
- Get data p2p

© 2025 University of Melbourne

Step 3: Design.
Choose an approach

Waterfall?

- Unwise to lock in, say, API before an overall design

Agile?

- Can’t have “the client” embedded in development

We will start with very limited functionality, and build up

© 2025 University of Melbourne

Phase 1: Trivial example

In groups, design a protocol to support one single function that

- Takes one 64-bit integer as an argument
- Returns one 64-bit integer as the value

Go back to the design decisions slide

© 2025 University of Melbourne

© 2025 University of Melbourne

Building up

- Extend the protocol to support multiple functions
  - How are functions identified?
  - Change in API (“services”)?
  - Change in message format?
  - New error conditions?

© 2025 University of Melbourne

Building up

- Extend the protocol to support multiple functions
- Repeat for functions of up to 32 integers
- Repeat for functions of a complex mix of data types
- Repeat for functions that return a struct
- Repeat for “stateful” functions

© 2025 University of Melbourne

Deployment questions

- How finished should it be before it is deployed?
- What happens when you release a new specification?
- How can you test thoroughly before public release?

© 2025 University of Melbourne

Summary

- Protocol design is iterative
  - Like software design
- Can be broken into subtasks:
  - What services should it offer?
  - What interactions are necessary?
  - How should data be represented?
  - What if things go wrong?
  - What security risks are there?

© 2025 University of Melbourne